



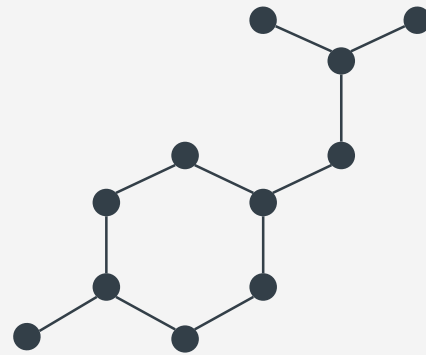
Universität
Münster

Graph Neural Networks and Geometric Deep Learning

Stefan Georgiev

Graph Neural Networks

graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$

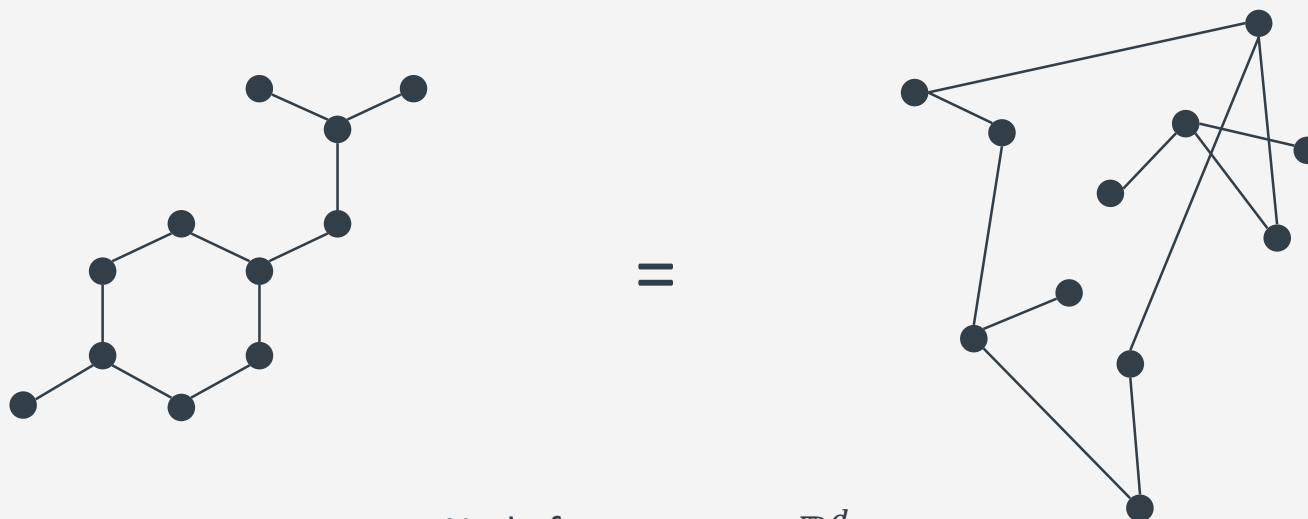


vertex (node) $v_i \in \mathcal{V}$

edge $e_{ij} \in \mathcal{E}$

neighbours $\mathcal{N}_i = \{v_j | (v_i, v_j) \in \mathcal{E}\}$

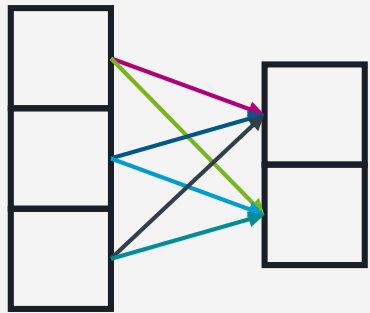
Graph Neural Networks



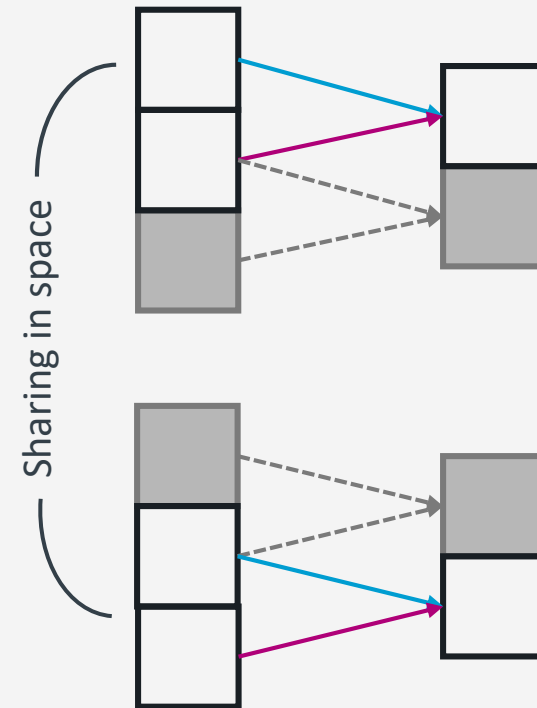
Node features: $v_i \in \mathbb{R}^{d_v}$
Edge features: $e_{i,j} \in \mathbb{R}^{d_e}$

Isomorphism

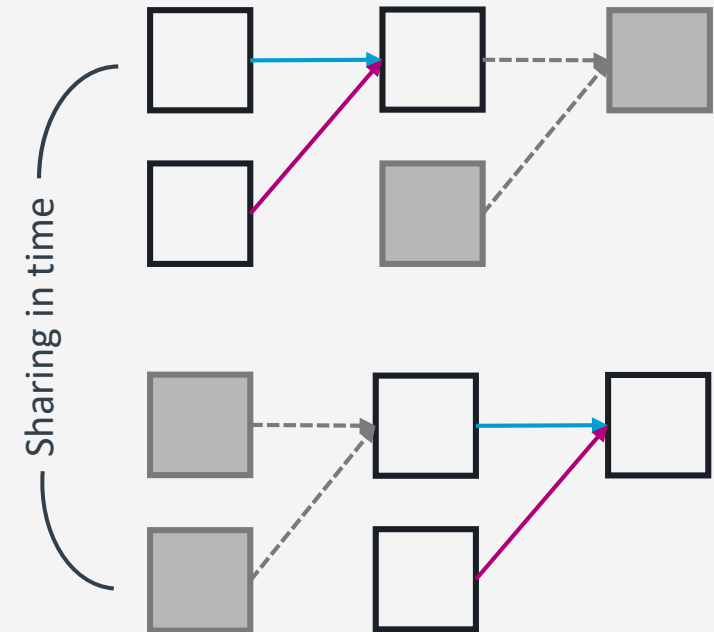
The One True Architecture?



Fully connected

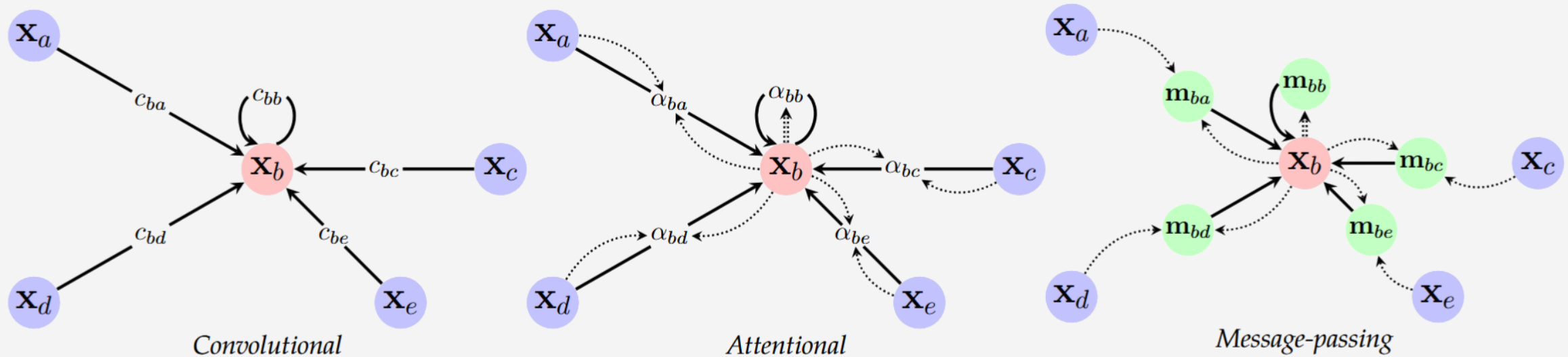


Convolutional



Recurrent

The three “flavours” of GNN layers



Expressive power increases, but more complex operations are required and less interpretable

*Also a higher risk of overfitting, as complex architectures have more parameters

Message Passing

The fundamental building block of GNNs.

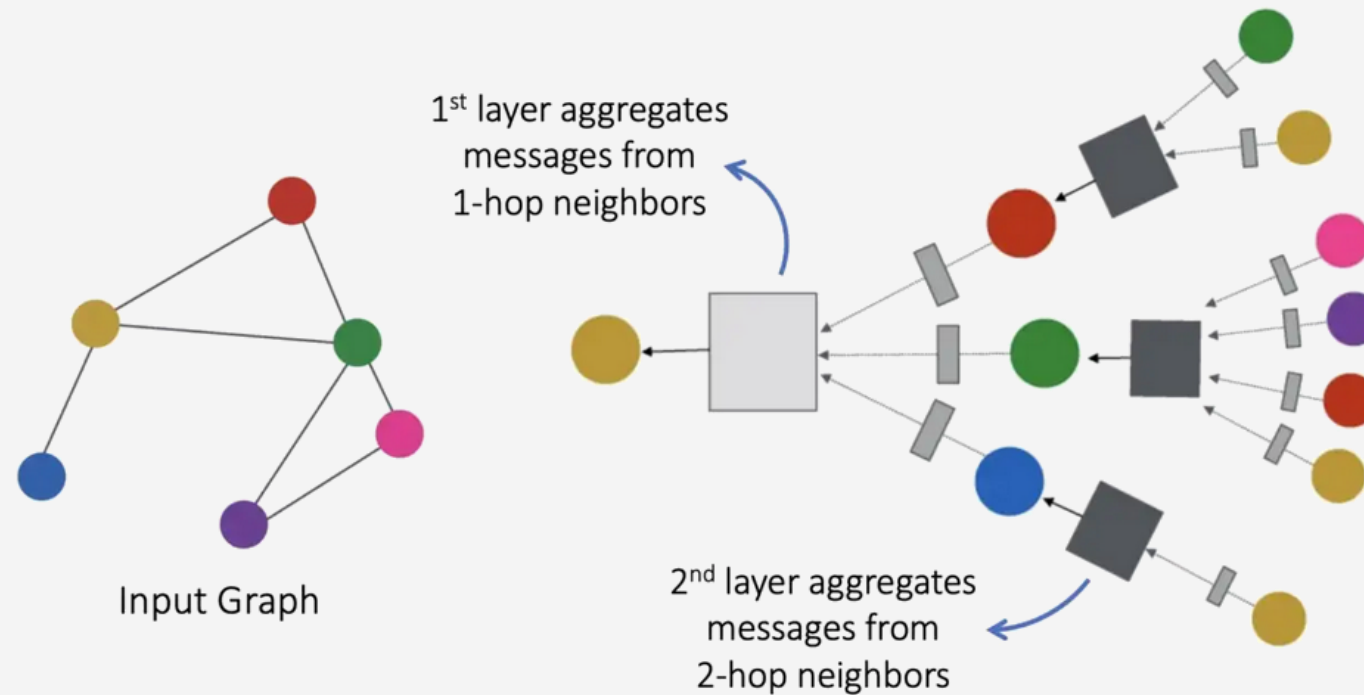
$$h_i^{(l+1)} = \tau \left(h_i^{(l)}, \bigoplus_{j \in \mathcal{N}_i} \mu \left(h_i^{(l)}, h_j^{(l)}, e_{ij} \right) \right)$$

Here $\bigoplus$ is a permutation-invariant function, e.g. sum or mean.

e.g.:
$$h_i^{(l+1)} = \tau \left(W^{(l+1)} \left(h_i^{(l)} + \sum_j h_j^{(l)} \right) \right)$$

Edge features can also be incorporated, message passing on the (directed) edges is possible, edge updates are also possible.

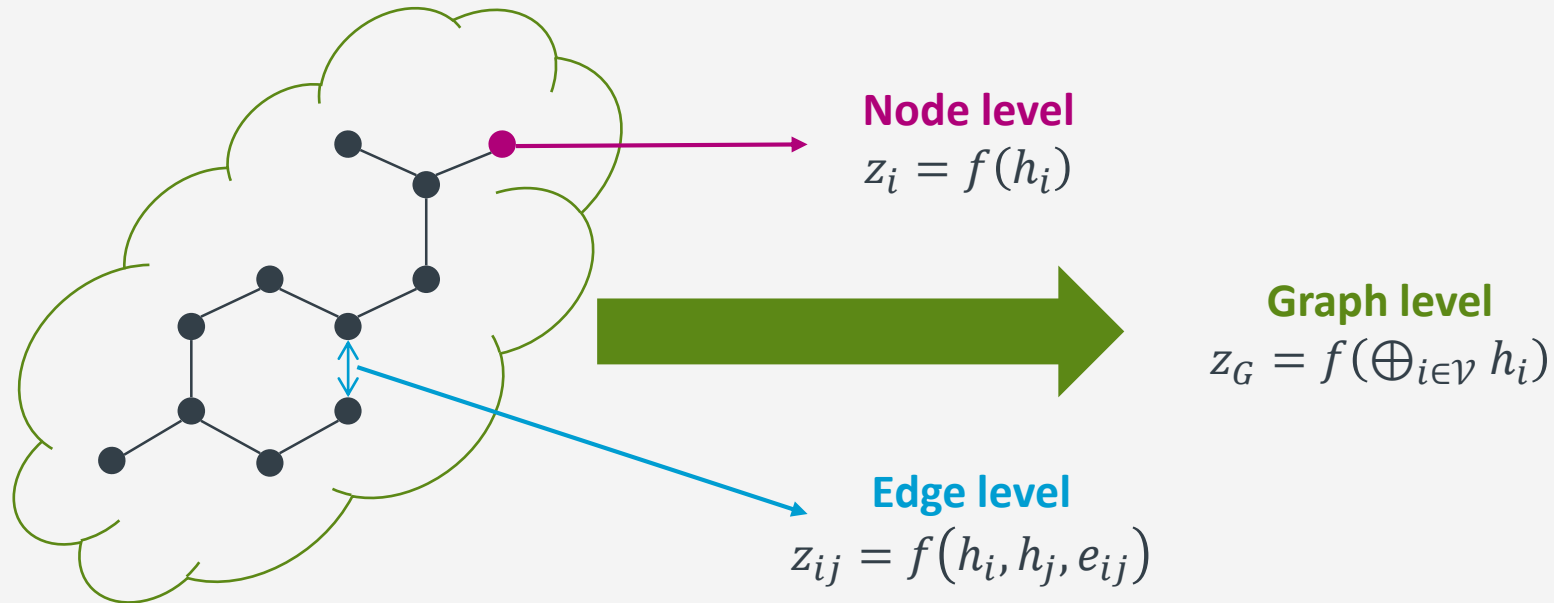
Message Passing



Predictions

Can be made on the **node** level, on the **edge** level, or on the **graph** level.

For graph-level predictions, a permutation-invariant pooling operation needs to take place beforehand.



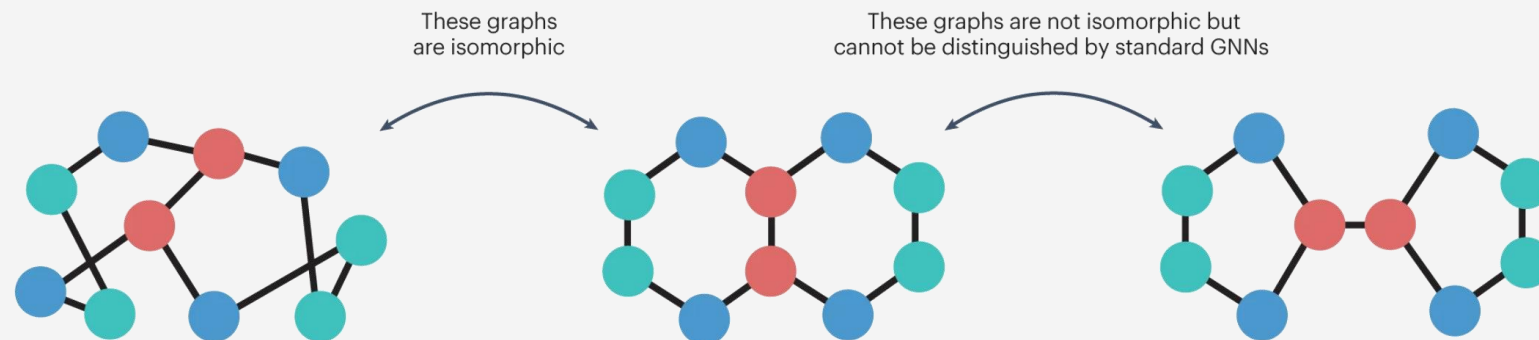
Expressivity

Expressivity analysis looks at which sets of functions a model is able to learn.

An architecture should be chosen with an expressivity that captures the patterns critical for the task. A too high expressivity can lead to overfitting and poor generalization!

Weisfeiler-Leman test

b

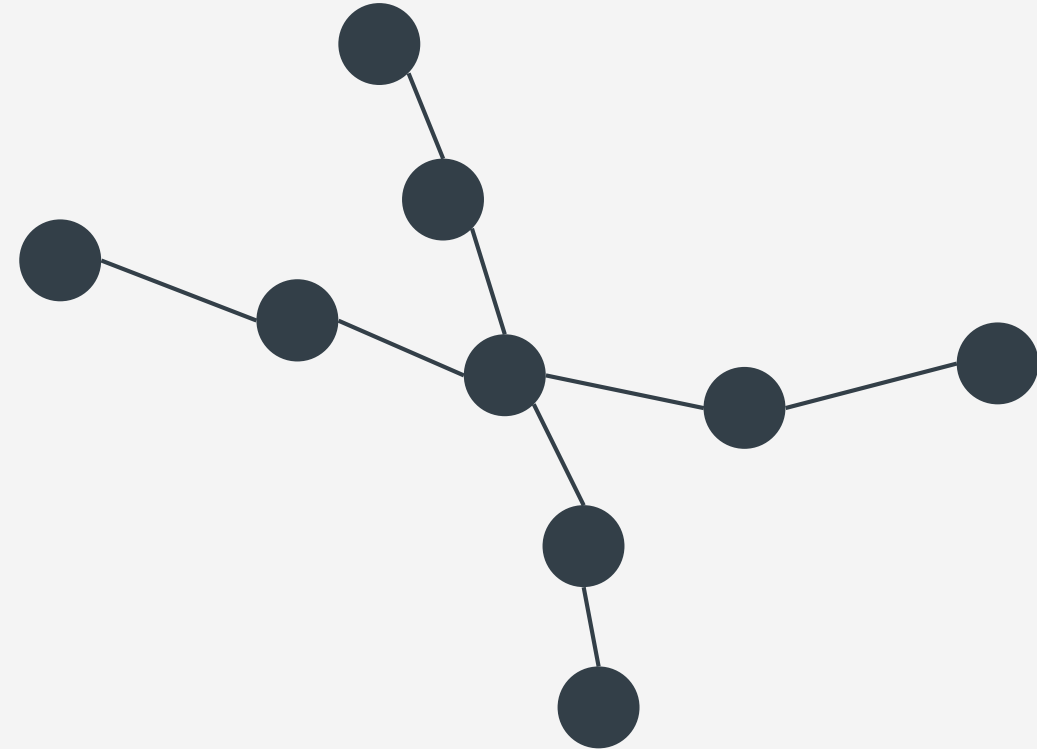
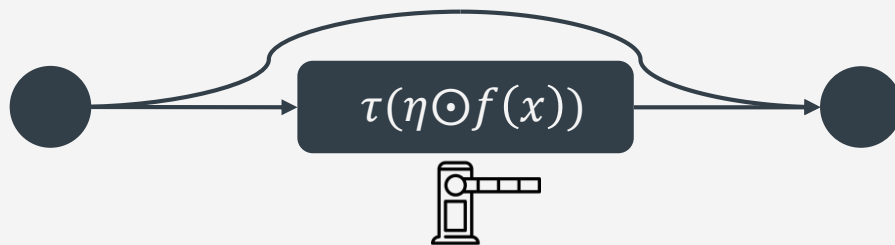


Some Problems of GNNs

oversmoothing

Potential remedies:

- skip connections
- gating mechanisms

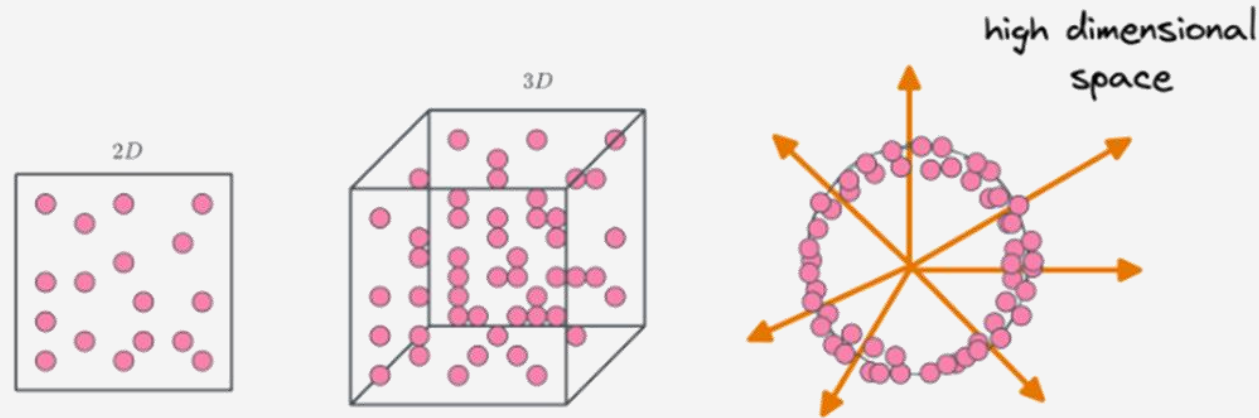


oversquashing (bottlenecking)

Potential remedies:

- adding a virtual global node
- rewiring the graph

Why do we want to restrict the functional space?



Curse of dimensionality: The number of samples needed grows exponentially

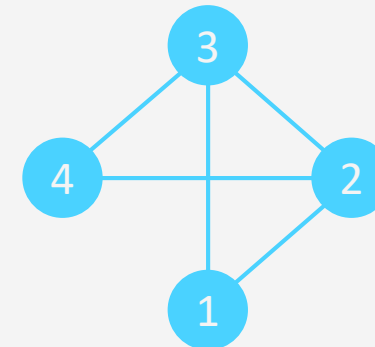
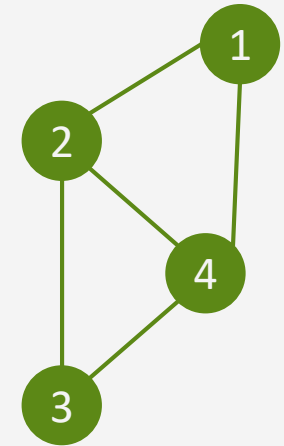
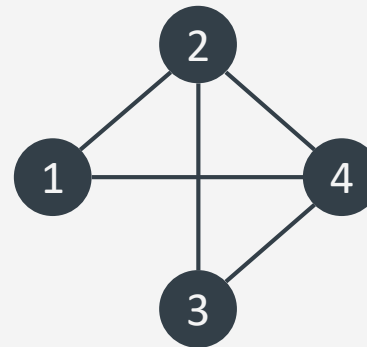
The answer: Geometry!

Apply *assumptions* about the **geometry** of the data → **inductive bias**

Restrict the possible functions to only the ones that respect the given geometry

Examples:

- CNNs: Translation invariance
- GNNs: Isomorphism, Permutation equivariance



The Space of Signals on a Geometric Domain

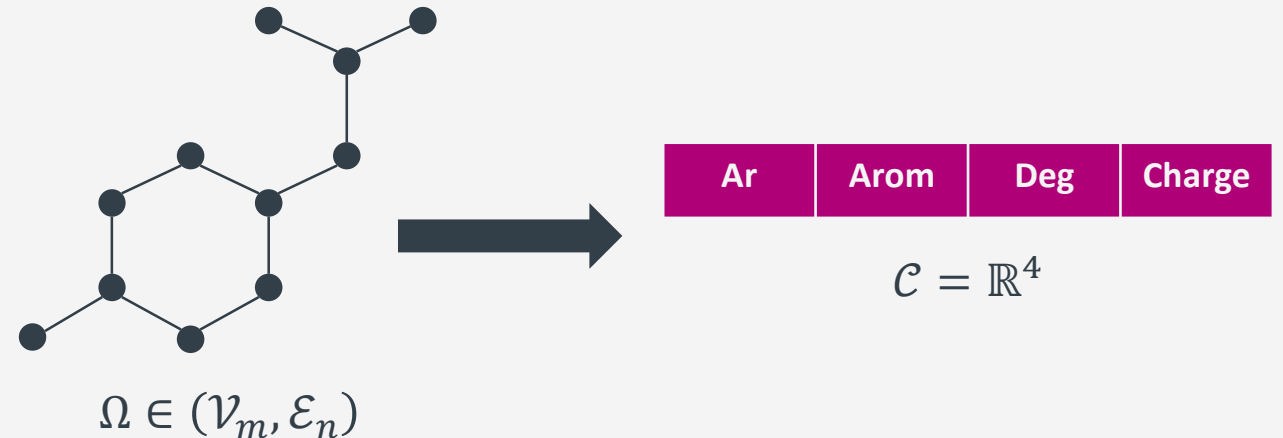
A formalization of the term *featurization*.

A *signal* on Ω is a function $x : \Omega \rightarrow \mathcal{C}$ where:

- Ω is the domain (e.g. a set of nodes)
- $\mathcal{C}$ is a vector space, whose dimensions are called *channels*

The space of $\mathcal{C}$ -valued signals on Ω is defined as $\mathcal{X}(\Omega, \mathcal{C}) = \{x : \Omega \rightarrow \mathcal{C}\}$

For *discrete* Ω signals are represented as *matrices*: $X \in \mathbb{R}^{|\Omega| \times \dim \mathcal{C}}$



Abstract groups

A group $(G, \cdot)$ is a **set of elements** G equipped with a **group product** $\cdot$, a binary operator, that satisfies the following four axioms:

- **Closure:** Given two elements g and h of G , the product $g \cdot h$ is also in G
- **Associativity:** For $g, h, i \in G$ the product $\cdot$ is associative: $g \cdot (h \cdot i) = (g \cdot h) \cdot i$
- **Identity:** There exists an identity element $e \in G$ such that $e \cdot g = g \cdot e = g$ for any $g \in G$
- **Inverse:** For each $g \in G$ there exists an inverse element $g^{-1} \in G$ such that $g \cdot g^{-1} = g^{-1} \cdot g = e$

Affine groups $G = \mathbb{R}^d \rtimes H$

Affine groups are semi-direct groups of some group H with an action on $\mathbb{R}^d$ from which we derive the following group product and inverse

$$g \cdot g' = (\mathbf{x}, h) \cdot (\mathbf{x}', h') = (h \cdot \mathbf{x}' + \mathbf{x}, h \cdot h')$$

$$g^{-1} = (-h^{-1} \cdot \mathbf{x}, h^{-1})$$

with group elements $g = (\mathbf{x}, h)$, $g' = (\mathbf{x}', h')$.

Representations

An n -dimensional real representation of a group G is a map $\rho : G \rightarrow \mathbb{R}^{n \times n}$, assigning to each $g \in G$ an invertible matrix $\rho(g)$ and satisfying:

$$\rho(gh) = \rho(g)\rho(h), \forall g, h \in G$$

A fundamental object in representation theory and geometric DL.

$\Rightarrow$ express the action of g in the domain of linear algebra 😊

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Symmetry

A **symmetry** is a transformation of an object that leaves it **unchanged**.

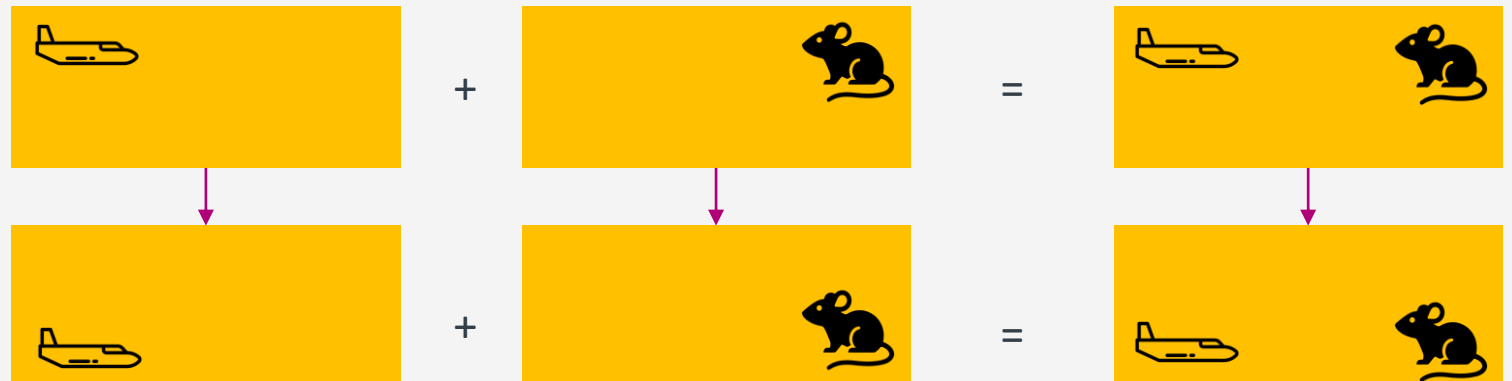
This defines some properties:

- The **identity** transformation is always a symmetry.
- The **composition** of two symmetry transformations is also a symmetry.
- Given any symmetry, it must be **invertible**.
- Its **inverse** is also a symmetry.

⇒ **Symmetries are groups!**

Linearity of the group action!

$$(gx)u = x(g^{-1}u)$$

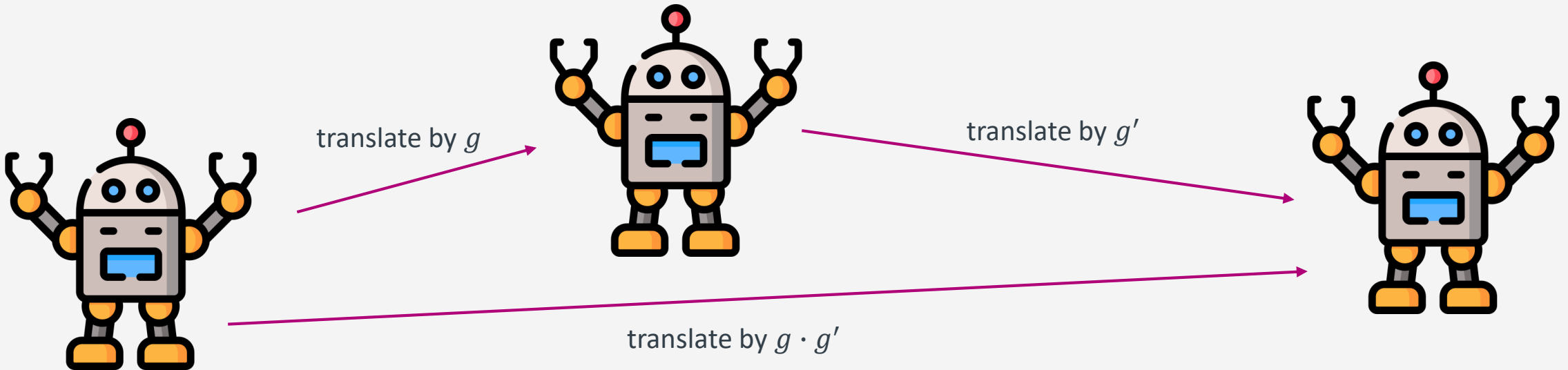


Translation group $(\mathbb{R}^2, +)$

The translation group consists of all possible translations (here, in $\mathbb{R}^2$) and is equipped with the **group product** and **group inverse**:

$$g \cdot g' = (x + x')$$

$$g^{-1} = (-x)$$



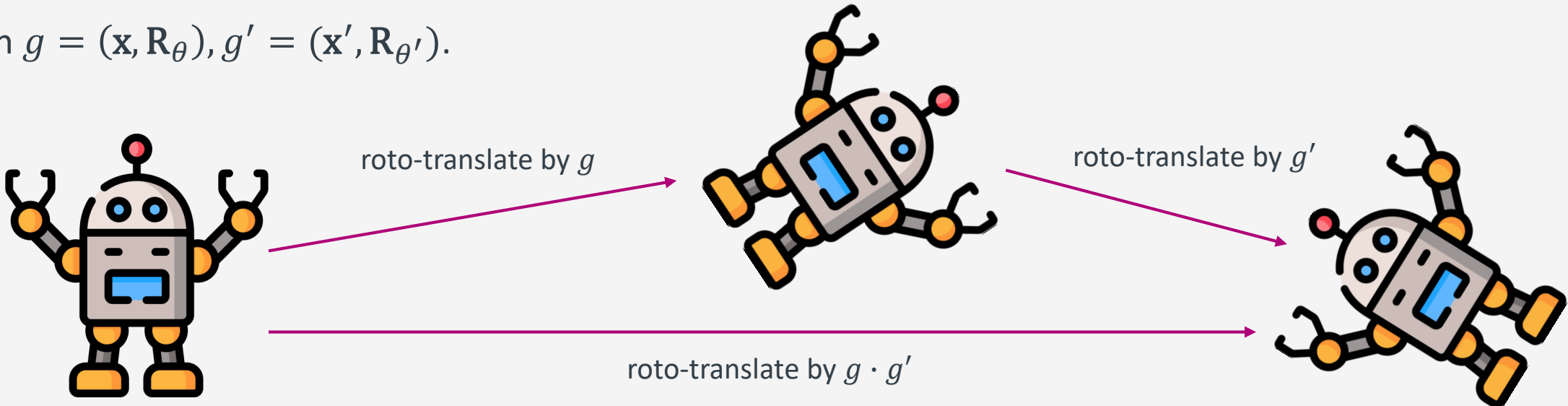
Roto-translation group $SE(2)$

The group $SE(2) = \mathbb{R}^2 \rtimes SO(2)$ consists of the **coupled** space $\mathbb{R}^2 \times S^1$ of translation vectors in $\mathbb{R}^2$ and rotations in $SO(2)$ (orientations in S^1) and is equipped with the group product and the inverse:

$$g \cdot g' = (\mathbf{x}, \mathbf{R}_\theta) \cdot (\mathbf{x}', \mathbf{R}_{\theta'}) = (\mathbf{R}_\theta \mathbf{x}' + \mathbf{x}, \mathbf{R}_{\theta+\theta'})$$

$$g^{-1} = (-\mathbf{R}_\theta^{-1} \mathbf{x}, \mathbf{R}_\theta^{-1})$$

with $g = (\mathbf{x}, \mathbf{R}_\theta)$, $g' = (\mathbf{x}', \mathbf{R}_{\theta'})$.



Interpreting Architectures through the Eyes of GDL

Architecture	Domain Ω	Symmetry group G
<i>CNN</i>	Grid	Translation
<i>Spherical CNN</i>	Sphere / $SO(3)$	Rotation $SO(3)$
<i>Intrinsic / Mesh CNN</i>	Manifold	Isometry $Iso(\Omega)$ / Gauge symmetry $SO(2)$
<i>GNN</i>	Graph	Permutation Σ_n
<i>Deep Sets</i>	Set	Permutation Σ_n
<i>Transformer</i>	Complete Graph	Permutation Σ_n
<i>LSTM</i>	1D Grid	Time warping

Molecules – Relevant symmetries

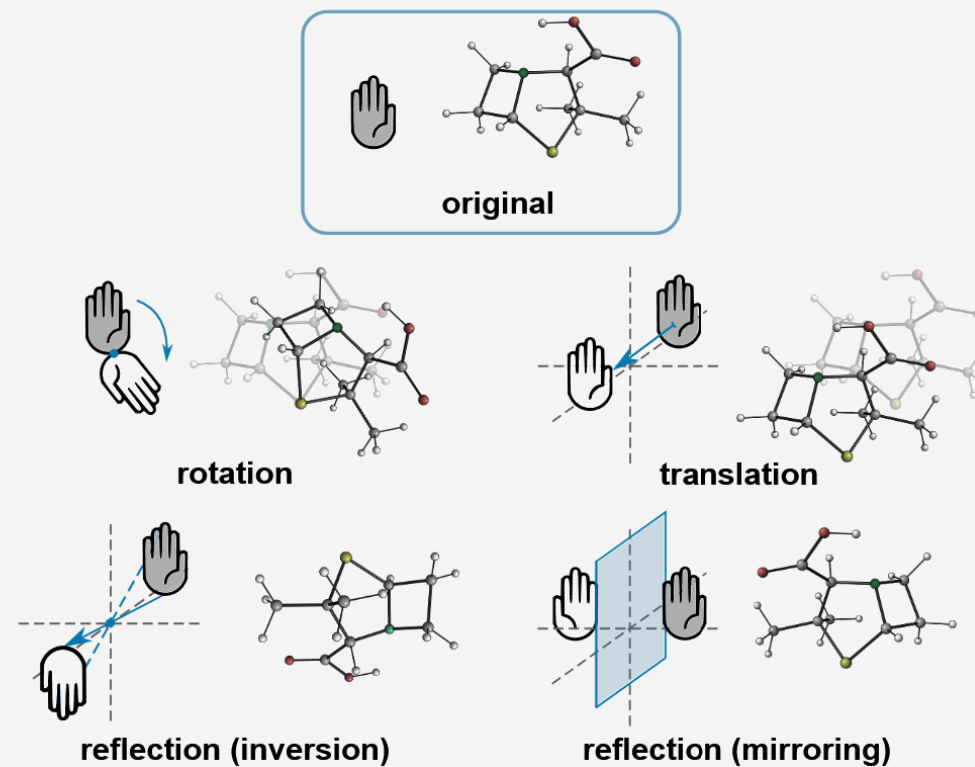
$SO(3)$ – rotation group

T - translation group

$SE(3)$ – roto-translation group

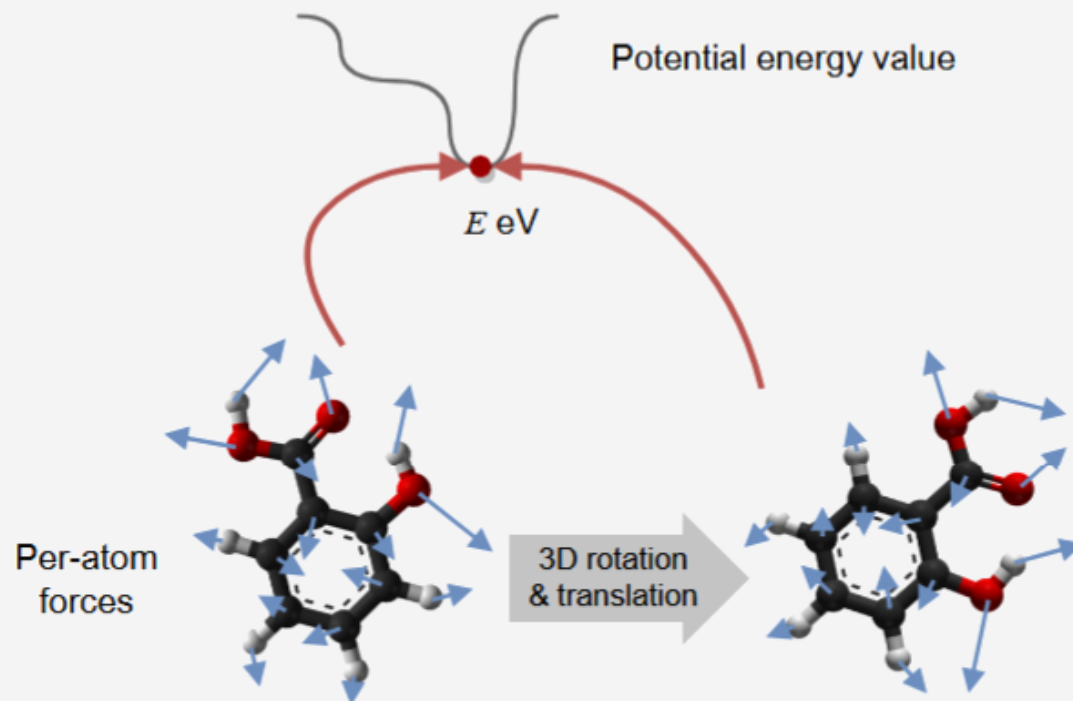
$O(3)$ – rotations and reflections

$E(3)$ – translations, rotations and reflections



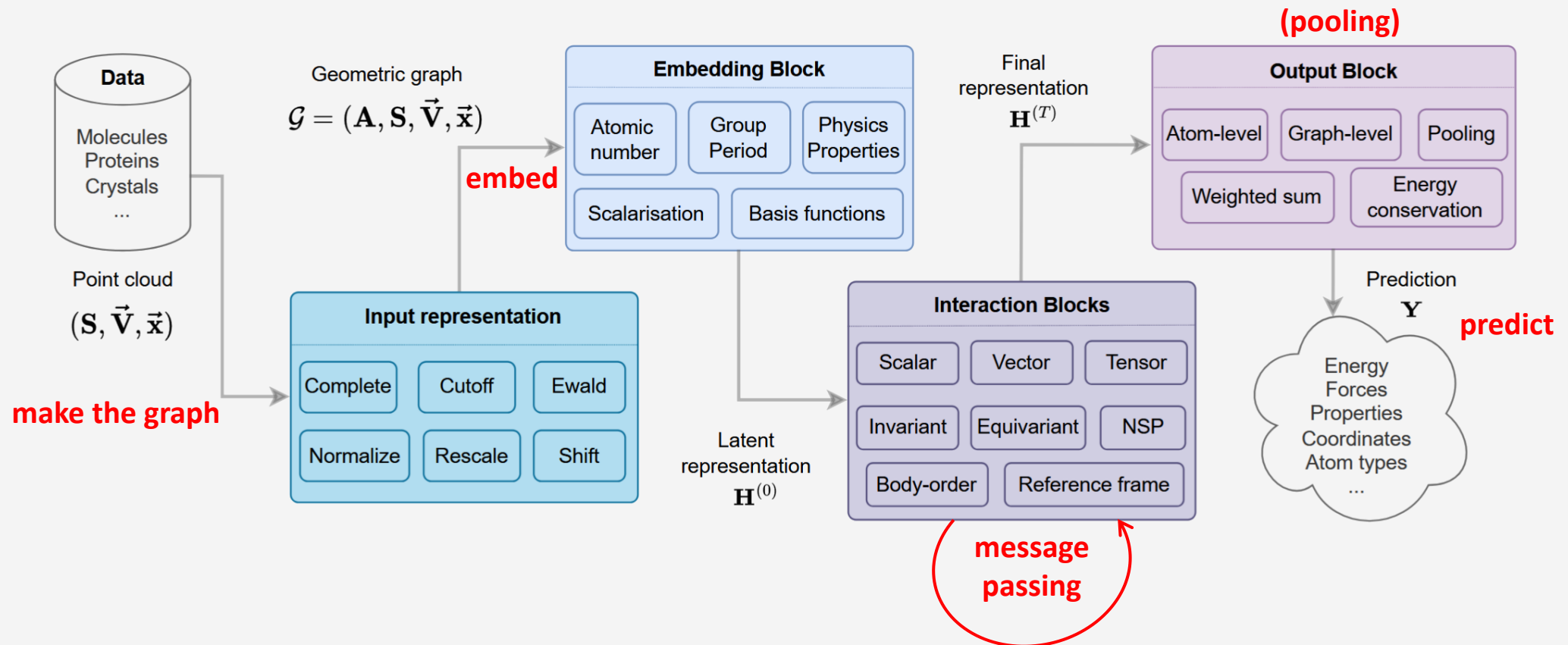
Invariance and Equivariance

A function $f : \mathcal{X}(\Omega) \rightarrow \mathcal{Y}$ is G -invariant if $f(\rho(g)x) = f(x)$ for all $g \in G$, i.e. the output is not affected by the group action on the input

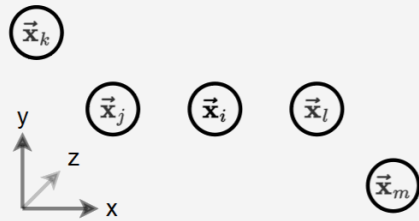


A function $f : \mathcal{X}(\Omega) \rightarrow \mathcal{Y}$ is G -equivariant if $f(\rho(g)x) = \rho(g)f(x)$ for all $g \in G$, i.e. applying the group action on the input affects the output in the same way

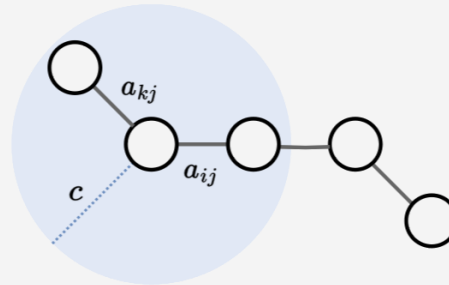
Deep Learning on Geometric Graphs



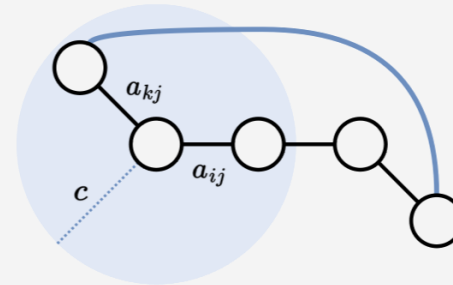
Geometric Graphs from Point Clouds



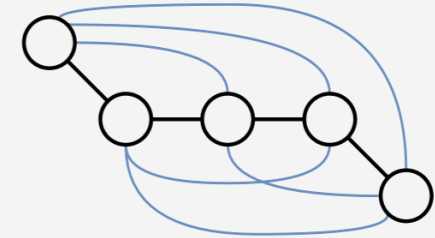
3D point cloud



Smoothed cutoff graph



Long-range connections



Complete graph

$$A_{ij} = \begin{cases} 1 & \text{if } d_{ij} \leq c \\ 0 & \text{otherwise} \end{cases}$$

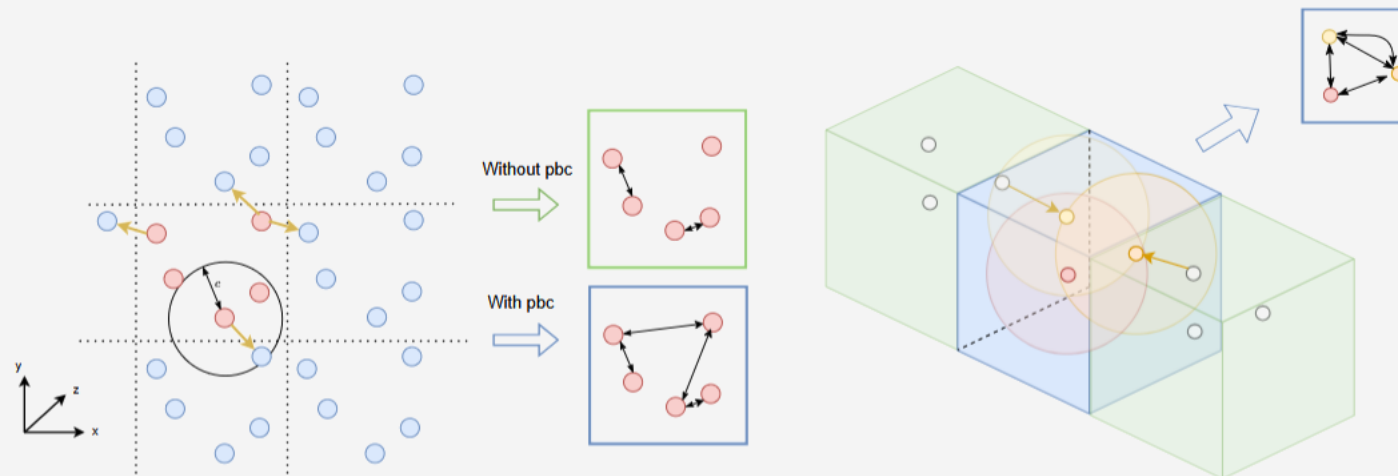
cutoff graph

$$d_{ij} = \|\vec{x}_i - \vec{x}_j\|$$

$$A_{ij} = \begin{cases} \frac{1}{2} \left(\cos \left(\frac{\pi d_{ij}}{c} \right) + 1 \right) & \text{if } d_{ij} \leq c \\ 0 & \text{otherwise} \end{cases}$$

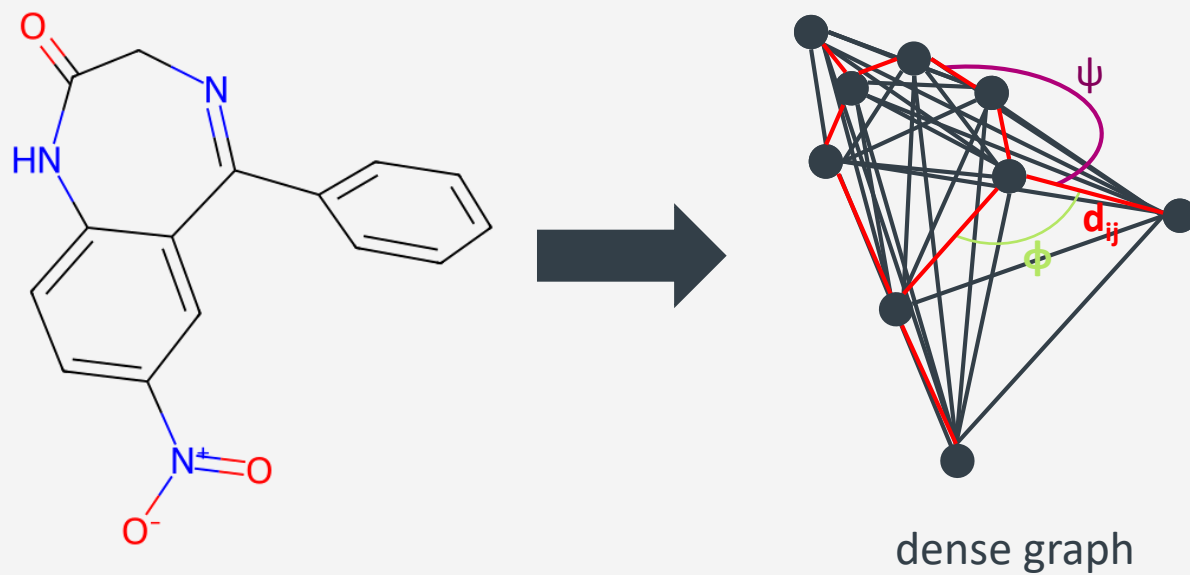
smooth cutoff graph
→ useful for e.g. predictions on MD simulations

Geometric Graphs from Point Clouds



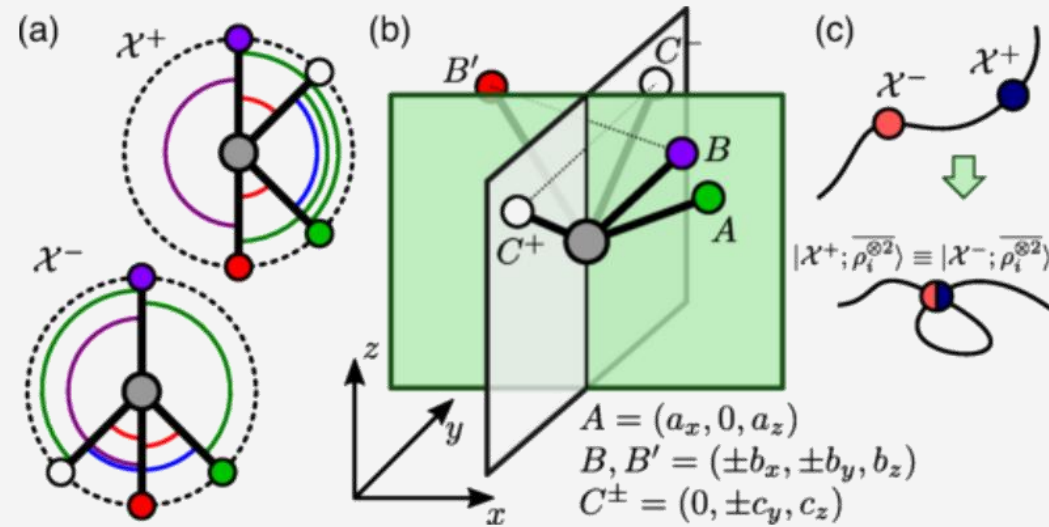
periodic boundary conditions

$SE(3)$ -Invariance



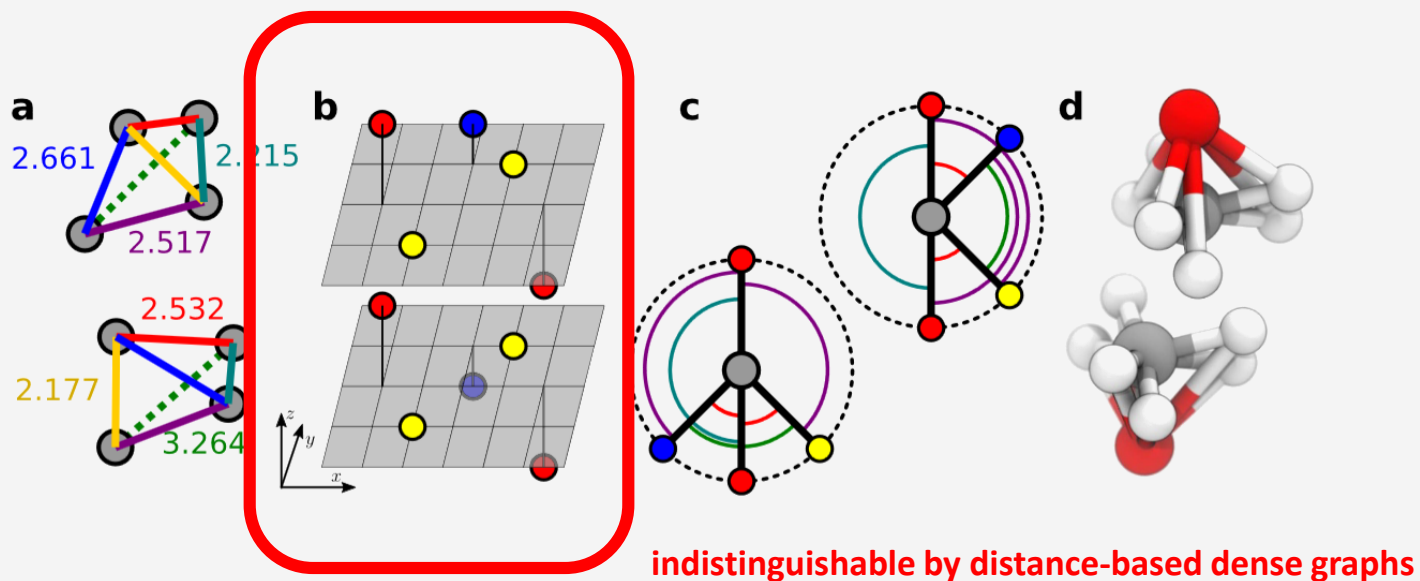
complete?

$SE(3)$ -Invariance

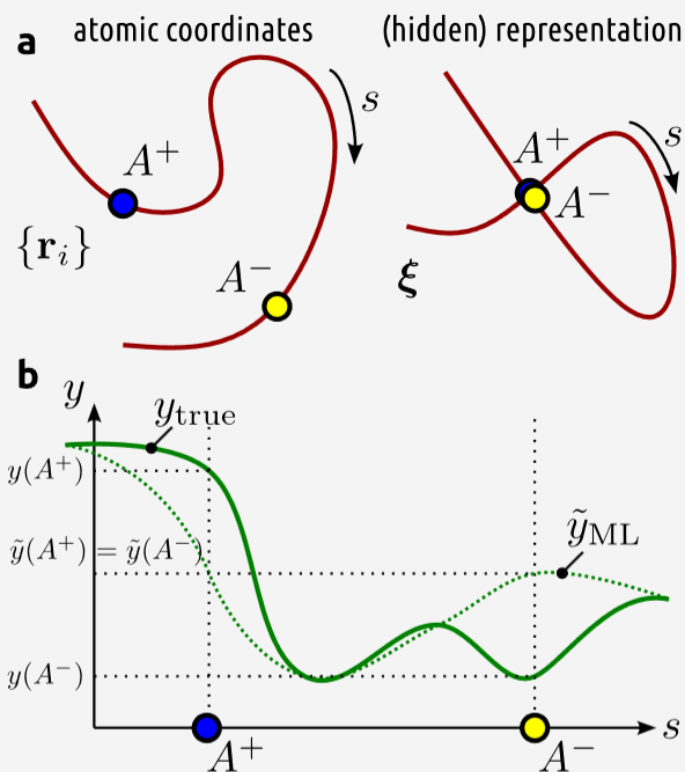


indistinguishable by graphs based on angles

$SE(3)$ -Invariance



$SE(3)$ -Invariance



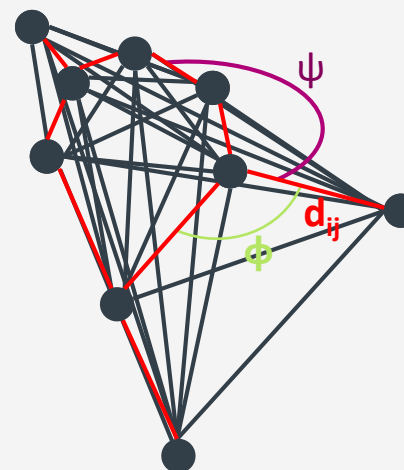
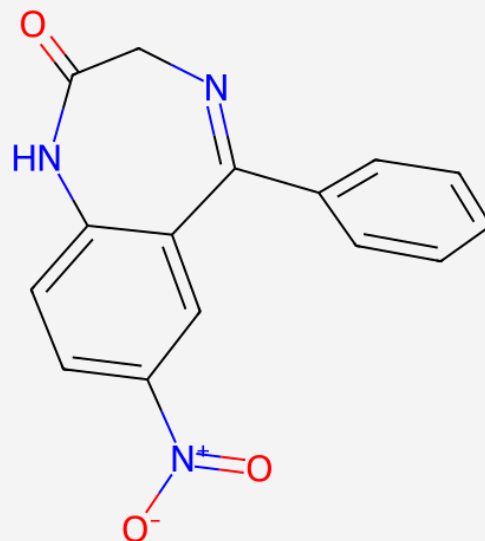
Wrong function is learned

$SE(3)$ -Invariance



Benefits:

- easy to make (normal GNNs)
- will work well for some datasets and questions (e.g. MD17, QM9)



dense graph



Problems:

- not universally expressive
- combinatorial explosion
- impractical for big molecules (exception: transformers)
- limited scalars

Equivariance

Let the model decide its own scalars.

When we want to incorporate information from tensors of rank > 0 or such are prediction targets.

*(Are universal approximators, when the graph is fully connected **and** using infinite rank tensors)

Rules of the Game



Perform *diligent accounting* of how each hidden feature in each layer needs to transform in order to remain equivariant!

1. Each feature has a **type*** that tells us how it transforms under symmetry.
2. When **adding** features component-wise, we need to only add features of the **same type** in order to ensure that the sum retains this same type.
3. We can **multiply** features of **different types**, but we need to keep track of how the product transforms. For this, the *tensor product* is used.
4. In ML, **non-linearities** are usually understood to be applied component-wise, which breaks equivariance for all but scalars due to **(2)** and **(3)**. Therefore, we need to **apply them to scalars** and then combine these scalars with the other features.

*Mathematically each type corresponds to different representations of the relevant group, i.e. $O(3)$ or $SO(3)$ here. In these slides, the two types we'll be discussing are tensors in Cartesian basis (Cartesian tensors) and tensors in spherical basis (spherical tensors).

Translation equivariance

Just subtract the center of mass / geometric center from all individual coordinates

$$\begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \\ x_4 & y_4 & z_4 \\ x_5 & y_5 & z_5 \end{bmatrix} = \begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \\ x_4 & y_4 & z_4 \\ x_5 & y_5 & z_5 \end{bmatrix} - [x_{\text{center}} \quad y_{\text{center}} \quad z_{\text{center}}]$$

⇒ we now only need to deal with rotations and reflections

WTF are (Cartesian) Tensors?

x

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

$$\begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \\ x_4 & y_4 & z_4 \\ x_5 & y_5 & z_5 \end{bmatrix}$$

$$\begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \\ x_4 & y_4 & z_4 \\ x_5 & y_5 & z_5 \end{bmatrix} \begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \\ x_4 & y_4 & z_4 \\ x_5 & y_5 & z_5 \end{bmatrix} \begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \\ x_4 & y_4 & z_4 \\ x_5 & y_5 & z_5 \end{bmatrix}$$

CS/ML:	scalar	vector	matrix	tensor
Maths:	rank 0 tensor	rank 1 tensor	rank 2 tensor	rank 3 tensor

Interaction (Multiplication) of Tensors?

$$a \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} ax \\ ay \\ az \end{bmatrix}$$

scalar times vector

$$\begin{bmatrix} a \\ b \\ c \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix} = ax + by + cz = s$$

inner product
(dot product)

$$a \cdot b$$

scalar multiplication

$$\|\vec{x}\| = \vec{x} \cdot \vec{x} = s$$

norm $\rightarrow$ dot product in disguise

$$\begin{bmatrix} a \\ b \\ c \end{bmatrix} \odot \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} ax \\ by \\ cz \end{bmatrix}$$

element-wise multiplication
(Hadamard product)

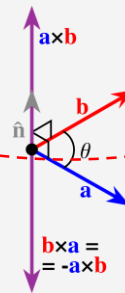
!!breaks symmetry:

$$R(\vec{u} \odot \vec{v}) \neq R\vec{u} \odot R\vec{v}$$

(in general)

$$\begin{bmatrix} a \\ b \\ c \end{bmatrix} \times \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} bz - cy \\ cx - az \\ ay - bx \end{bmatrix} = \begin{bmatrix} g \\ h \\ i \end{bmatrix}$$

cross product $\rightarrow$ pseudovector!



$$\begin{bmatrix} a \\ b \\ c \end{bmatrix} \otimes \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} ax & ay & az \\ bx & by & bz \\ cx & cy & cz \end{bmatrix}$$

tensor product
(outer product)

preserves symmetry, while
generating new types

Example - Scalar-Vector GNNs

general form:

$$\mathbf{m}_{ij}^{(t)}, \vec{\mathbf{m}}_{ij}^{(t)} := \text{Msg}\left(\mathbf{s}_i^{(t)}, \mathbf{s}_j^{(t)}, \vec{\mathbf{v}}_i^{(t)}, \vec{\mathbf{v}}_j^{(t)}, \vec{\mathbf{x}}_{ij}\right)$$

$$\mathbf{s}_i^{(t+1)}, \vec{\mathbf{v}}_i^{(t+1)} := \text{Upd}\left(\left(\mathbf{s}_i^{(t)}, \vec{\mathbf{v}}_i^{(t)}\right), \bigoplus_{j \in \mathcal{N}_i} (\mathbf{m}_{ij}^{(t)}, \vec{\mathbf{m}}_{ij}^{(t)})\right)$$

$$\mathbf{m}_i := \mathbf{f}_1(\mathbf{s}_i, \|\vec{\mathbf{v}}_i\|) + \sum_{j \in \mathcal{N}_i} \mathbf{f}_2(\mathbf{s}_i, \mathbf{s}_j, \|\vec{\mathbf{x}}_{ij}\|, \|\vec{\mathbf{v}}_j\|, \vec{\mathbf{x}}_{ij} \cdot \vec{\mathbf{v}}_j, \vec{\mathbf{x}}_{ij} \cdot \vec{\mathbf{v}}_i, \vec{\mathbf{v}}_i \cdot \vec{\mathbf{v}}_j)$$

$$\begin{aligned} \vec{\mathbf{m}}_i &:= \mathbf{f}_3(\mathbf{s}_i, \|\vec{\mathbf{v}}_i\|) \odot \vec{\mathbf{v}}_i + \sum_{j \in \mathcal{N}_i} \mathbf{f}_4(\mathbf{s}_i, \mathbf{s}_j, \|\vec{\mathbf{x}}_{ij}\|, \|\vec{\mathbf{v}}_j\|, \vec{\mathbf{x}}_{ij} \cdot \vec{\mathbf{v}}_j, \vec{\mathbf{x}}_{ij} \cdot \vec{\mathbf{v}}_i, \vec{\mathbf{v}}_i \cdot \vec{\mathbf{v}}_j) \odot \vec{\mathbf{v}}_j \\ &\quad + \sum_{j \in \mathcal{N}_i} \mathbf{f}_5(\mathbf{s}_i, \mathbf{s}_j, \|\vec{\mathbf{x}}_{ij}\|, \|\vec{\mathbf{v}}_j\|, \vec{\mathbf{x}}_{ij} \cdot \vec{\mathbf{v}}_j, \vec{\mathbf{x}}_{ij} \cdot \vec{\mathbf{v}}_i, \vec{\mathbf{v}}_i \cdot \vec{\mathbf{v}}_j) \odot \vec{\mathbf{x}}_{ij} \end{aligned}$$

Example – Scalar-Vector GNNs (Low Tensor Rank Models)

$$\mathbf{m}_i^{(t)} := \mathbf{s}_i^{(t)} + \sum_{j \in \mathcal{N}_i} \mathbf{f}_1(\mathbf{s}_j^{(t)}, \|\vec{x}_{ij}\|)$$

$$\vec{\mathbf{m}}_i^{(t)} := \vec{\mathbf{v}}_i^{(t)} + \sum_{j \in \mathcal{N}_i} \mathbf{f}_2(\mathbf{s}_j^{(t)}, \|\vec{x}_{ij}\|) \odot \mathbf{v}_j^{(t)} + \sum_{j \in \mathcal{N}_i} \mathbf{f}_3(\mathbf{s}_j^{(t)}, \|\vec{x}_{ij}\|) \odot \vec{x}_{ij}$$

$$\mathbf{s}_i^{(t+1)} := \mathbf{m}_i^{(t)} + \mathbf{f}_4(\mathbf{m}_i^{(t)}, \|\vec{\mathbf{m}}_i^{(t)}\|)$$

$$\mathbf{v}_i^{(t+1)} := \underbrace{\vec{\mathbf{m}}_i^{(t)} + \mathbf{f}_5(\mathbf{m}_i^{(t)}, \|\vec{\mathbf{m}}_i^{(t)}\|)}_{\text{e.g. gated non-linearity}} \odot \vec{\mathbf{m}}_i^{(t)}$$

e.g. gated non-linearity

Rotation of Cartesian Tensors

$$\vec{T}_{ijk}^{[3]} = \vec{v}_i \vec{v}_j \vec{w}_k = \vec{v}_i \otimes \vec{v}_j \otimes \vec{w}_k$$

Full notation:

$$\sum_{m=1}^3 \sum_{n=1}^3 R_{im} R_{jn} \vec{M}_{mn}^{[2]}$$

$$\sum_{l=1}^3 \sum_{m=1}^3 \sum_{n=1}^3 R_{il} R_{jm} R_{kn} \vec{T}_{lmn}^{[3]}$$

$$\sum_{i=1}^3 \sum_{j=1}^3 \sum_{k=1}^3 \sum_{l=1}^3 \sum_{m=1}^3 R_{ri} R_{sj} R_{tk} R_{ul} R_{vm} \vec{U}_{ijklm}^{[5]}$$

Einstein summation notation:

$$R_{im} R_{jn} \vec{M}_{mn}^{[2]}$$

$$R_{il} R_{jm} R_{kn} \vec{T}_{lmn}^{[3]}$$

$$R_{ri} R_{sj} R_{tk} R_{ul} R_{vm} \vec{U}_{ijklm}^{[5]}$$

Going Up and Down Ranks

Going up:

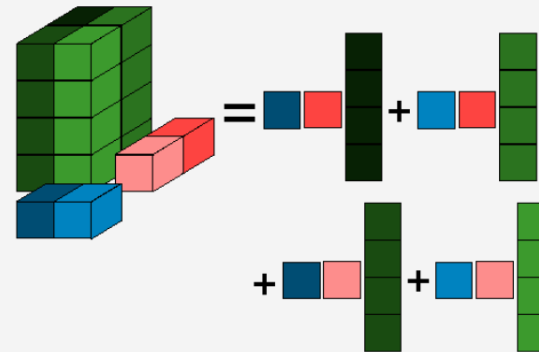
Tensor product ($\otimes$): multiplying two Cartesian tensors of ranks c_1 and c_2 yields a new Cartesian tensor of rank $c_1 + c_2$ with $3^{c_1+c_2}$ components.

Going down:

Contraction ($\mathfrak{C}$): a generalized dot product-like operation equivariantly generates tensors of lower ranks.

Tensor Contraction

$$T(u, v, \cdot) = \sum_{i,j} u_i v_j T_{i,j,\cdot}$$



$$\vec{V}^{[0]} = \vec{T}_n^{[1]} \vec{U}_n^{[1]}$$

$$\vec{V}^{[0]} = \mathfrak{C}_{(1:2)} [\vec{T}^{[1]} \otimes \vec{U}^{[1]}]$$

$$\vec{V}_{ijkl}^{[4]} = \vec{T}_{nmn}^{[3]} \vec{U}_{imjkl}^{[5]}$$

$$\vec{V}^{[4]} = \mathfrak{C}_{(1:3)(2:5)} [\vec{T}^{[3]} \otimes \vec{U}^{[5]}]$$

*only symmetric contractions shown here, asymmetric contractions would also need to be included in the model architecture

<https://medium.com/@theter/thinking-general-relativity-8-f05edfcfeddb>

Problems with Cartesian Tensors

Carry redundant information.

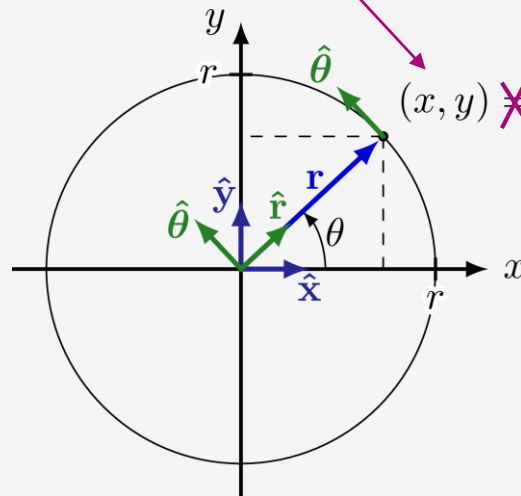
Very high memory cost in higher ranks.

Isn't there something more efficient?

The idea

Change of viewpoint; simplify the problem

Cartesian coordinates

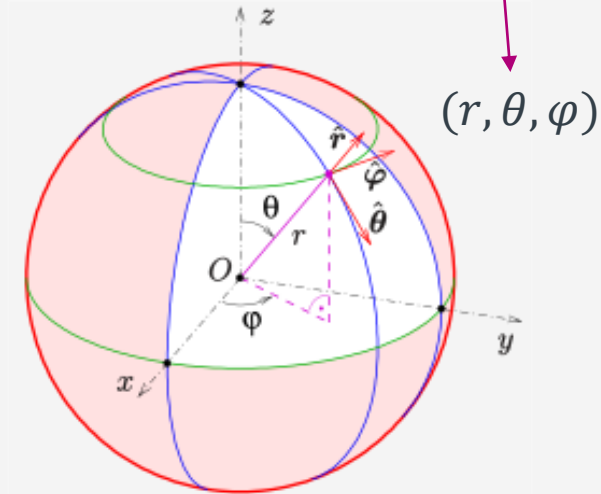


polar coordinates

$(x, y) \not\equiv (r; \theta)$

$$\begin{aligned} x &= r \cos \theta \\ y &= r \sin \theta \end{aligned}$$

spherical coordinates



radius r , polar angle θ , azimuthal angle φ

Set $r = 1$ for numerical stability in the EGNNs

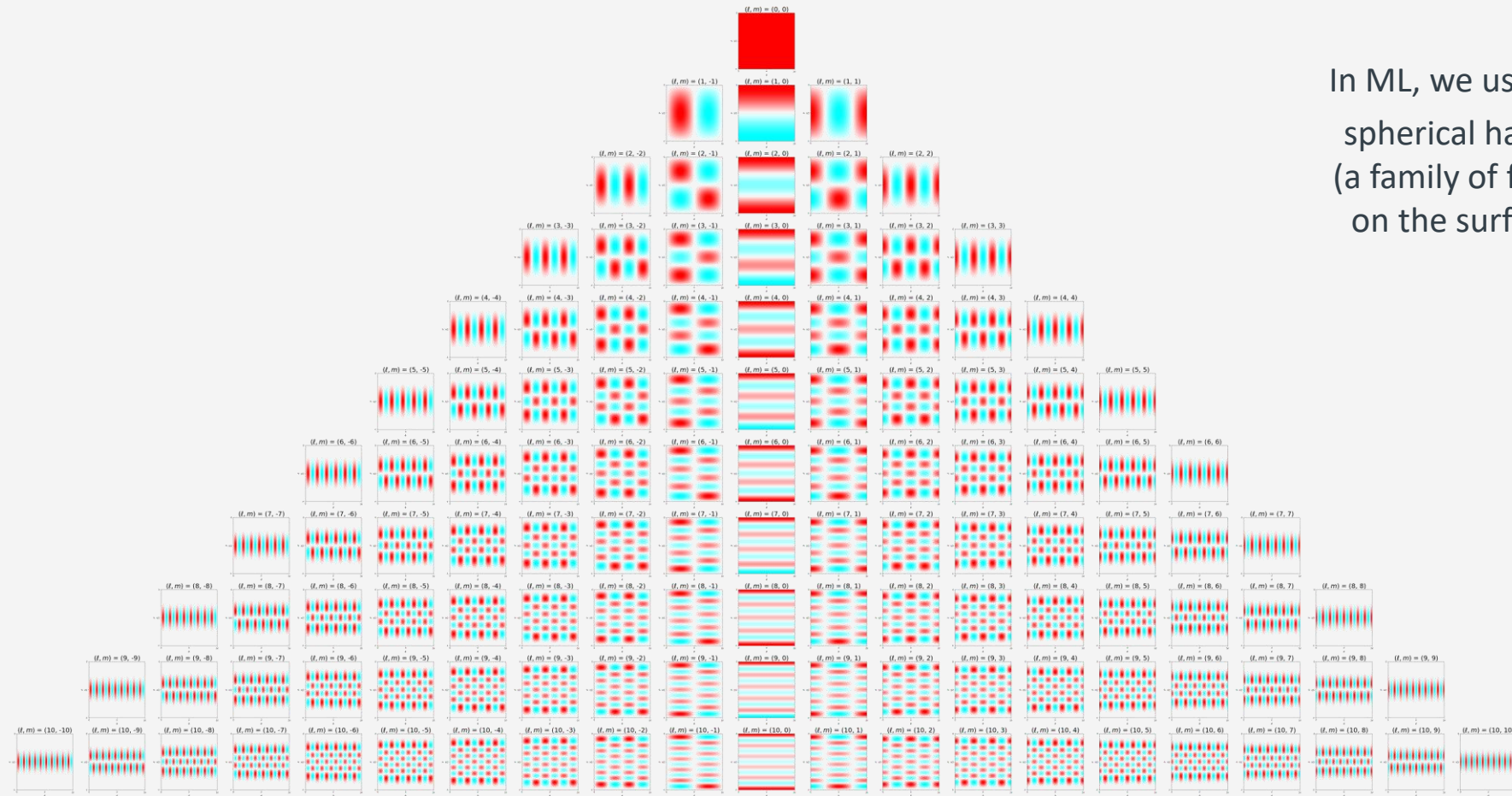
<https://tikz.net/circle/>; https://en.wikipedia.org/wiki/Spherical_coordinate_system

Spherical Harmonics – New Basis



In ML, we use the **real-valued** spherical harmonics $\vec{Y}_m^{(l)}(\hat{x})$ (a family of functions defined on the surface of a sphere)

Spherical Harmonics

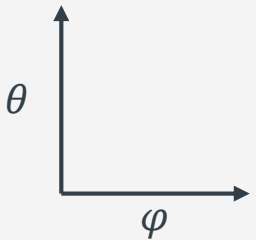


In ML, we use the **real-valued** spherical harmonics $\vec{Y}_m^{(l)}(\hat{x})$ (a family of functions defined on the surface of a sphere)

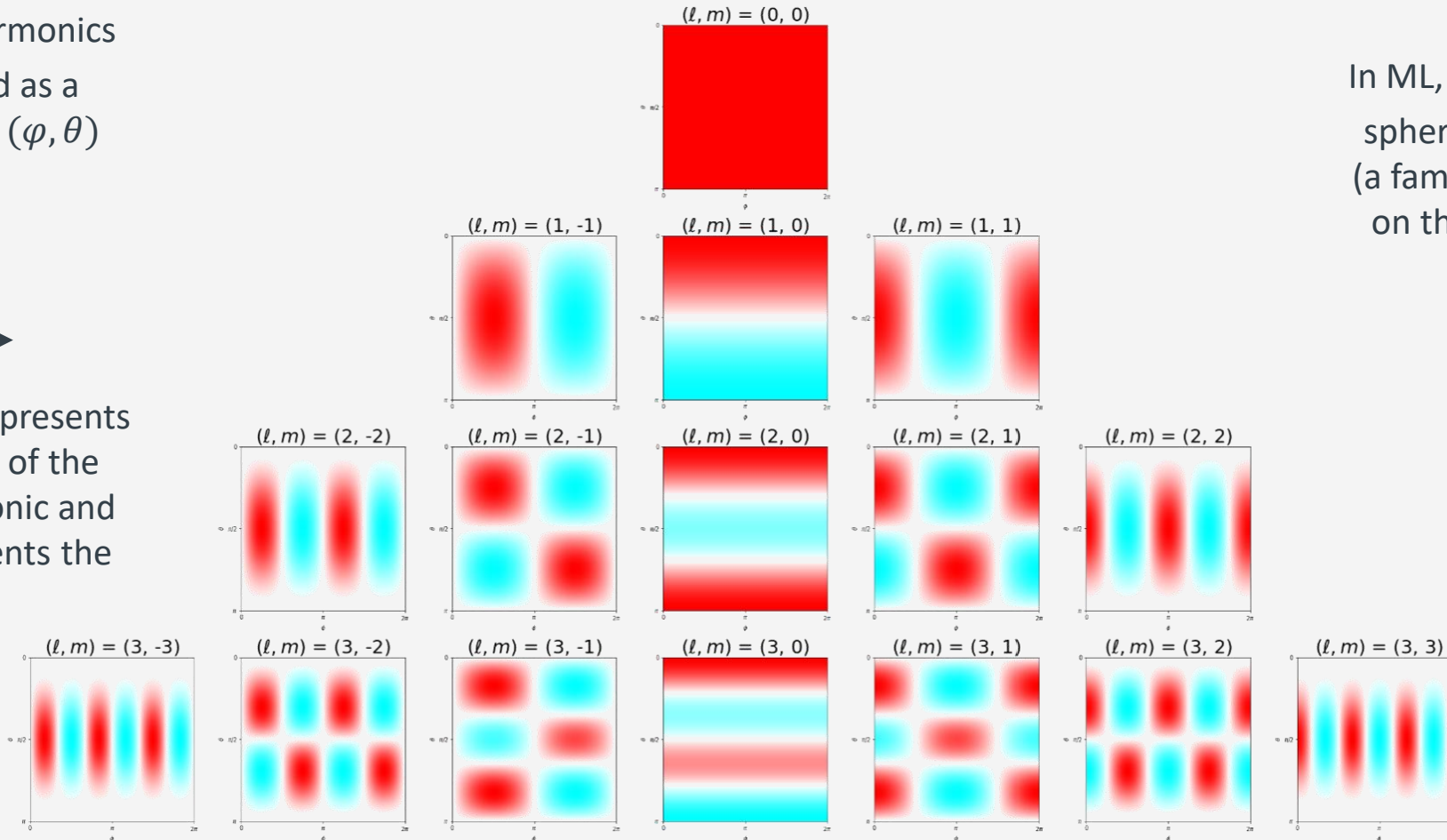
Spherical Harmonics

real spherical harmonics

$\vec{Y}_m^{(l)}(\hat{x})$ plotted as a
function of $\hat{x} = (\varphi, \theta)$

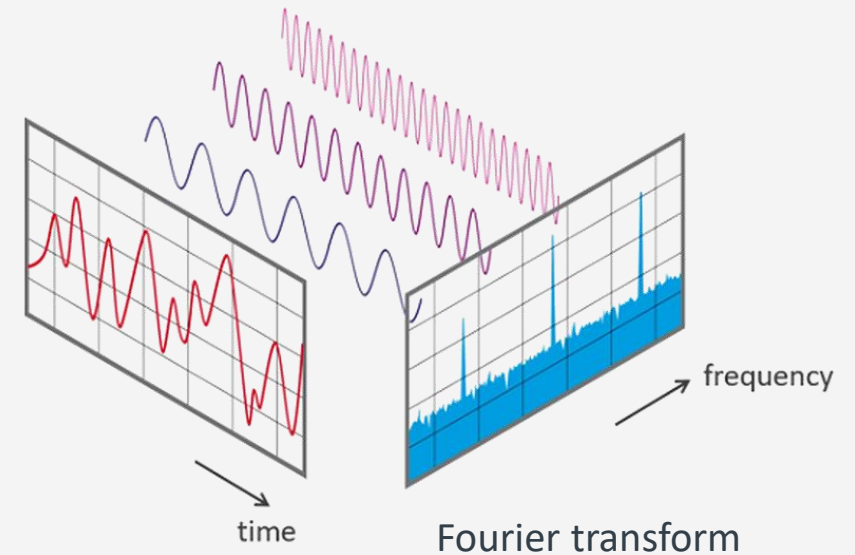
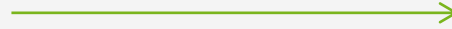
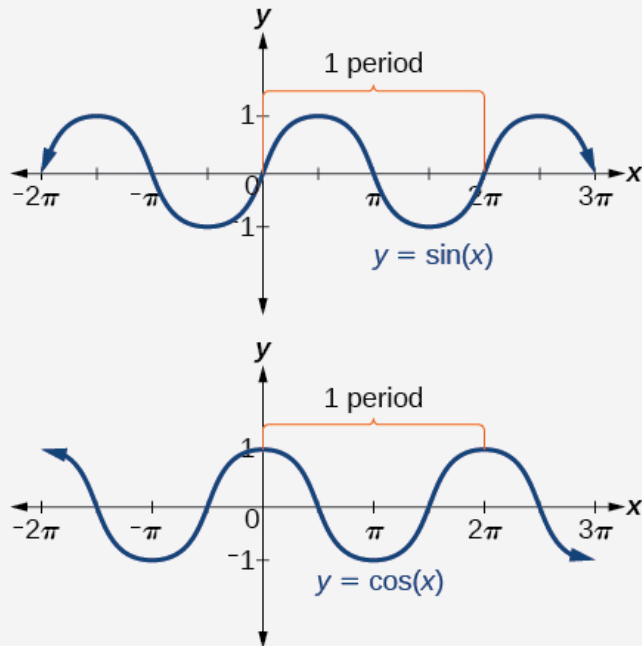


the saturation represents
the magnitude of the
spherical harmonic and
the hue represents the
sign



In ML, we use the **real-valued**
spherical harmonics $\vec{Y}_m^{(l)}(\hat{x})$
(a family of functions defined
on the surface of a sphere)

Periodicity – Comparison to Something We Know



⇒ Every smooth function on a sphere is decomposable to a sum of spherical harmonics, weighted by different coefficients. Therefore, spherical harmonics form a complete and orthonormal basis of such functions!

Irreps

All types can be decomposed into the fundamental irreducible representations (*irreps* for short).

Irreps are the smallest representations, which are similar to, and transform the same way as, the spherical harmonics.

The irreps of $SO(3)$ can be numbered by $l = 1, 2, 3, \dots \rightarrow$ *spherical tensor* types $\vec{T}^{(l)}$.

A spherical tensor of degree (order / angular frequency) l is $2l + 1$ -dimensional.

Decomposition

$l = 0$ corresponds to the trace of $\vec{T}^{[2]}$, therefore holds the same information as the dot product of the underlying vectors $\vec{v}_i, \vec{w}_j$ that made $\vec{T}^{[2]}$.

$$\vec{v}_i \cdot \vec{w}_j = 3\lambda_1$$

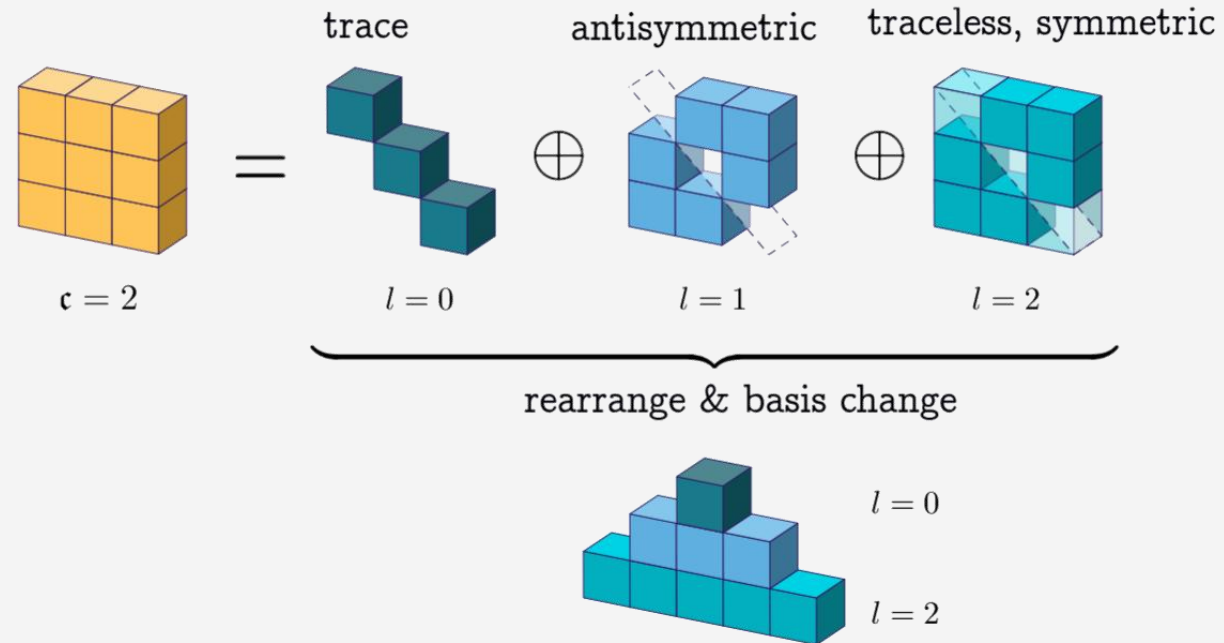
$l = 1$ is the asymmetric part and corresponds to the cross product space.

$$\vec{v}_i \times \vec{w}_j = \begin{bmatrix} \lambda_2 \\ \lambda_3 \\ \lambda_4 \end{bmatrix}$$

$l = 2$ corresponds to a traceless, symmetric matrix.

$$\begin{aligned} \vec{T}^{[2]} &= \begin{bmatrix} T_{xx} & T_{xy} & T_{xz} \\ T_{yx} & T_{yy} & T_{yz} \\ T_{zx} & T_{zy} & T_{zz} \end{bmatrix} \\ &= \frac{T_{xx} + T_{yy} + T_{zz}}{3} \begin{bmatrix} 1 & & \\ & 1 & \\ & & 1 \end{bmatrix} & (l = 0) \\ &+ \begin{bmatrix} & \frac{T_{xy} - T_{yx}}{2} & \frac{T_{xz} - T_{zx}}{2} \\ -\frac{T_{xy} - T_{yx}}{2} & & \frac{T_{yz} - T_{zy}}{2} \\ -\frac{T_{xz} - T_{zx}}{2} & -\frac{T_{yz} - T_{zy}}{2} & \end{bmatrix} & (l = 1) \\ &+ \begin{bmatrix} \frac{T_{xx}}{2} & \frac{T_{xy} + T_{yx}}{2} & \frac{T_{xz} + T_{zx}}{2} \\ \frac{T_{xy} + T_{yx}}{2} & T_{yy} & \frac{T_{yz} + T_{zy}}{2} \\ \frac{T_{xz} + T_{zx}}{2} & \frac{T_{yz} + T_{zy}}{2} & T_{zz} \end{bmatrix} - \frac{T_{xx} + T_{yy} + T_{zz}}{3} \begin{bmatrix} 1 & & \\ & 1 & \\ & & 1 \end{bmatrix} & (l = 2) \\ &= \frac{\lambda_1}{3} \begin{bmatrix} 1 & & \\ & 1 & \\ & & 1 \end{bmatrix} & (l = 0) \\ &+ \frac{\lambda_2}{2} \begin{bmatrix} & & \\ & -1 & \\ -1 & & 1 \end{bmatrix} + \frac{\lambda_3}{2} \begin{bmatrix} & & -1 \\ & 1 & \\ 1 & & -1 \end{bmatrix} + \frac{\lambda_4}{2} \begin{bmatrix} & & \\ -1 & & 1 \end{bmatrix} & (l = 1) \\ &+ \frac{\lambda_5}{3} \begin{bmatrix} 1 & & \\ & -1 & \\ & & 1 \end{bmatrix} + \frac{\lambda_6}{2} \begin{bmatrix} & & \\ 1 & & 1 \end{bmatrix} + \frac{\lambda_7}{2} \begin{bmatrix} & & 1 \\ & 1 & \\ 1 & & \end{bmatrix} \\ &+ \frac{\lambda_8}{2} \begin{bmatrix} & & \\ & 1 & \\ 1 & & \end{bmatrix} + \frac{\lambda_9}{3} \begin{bmatrix} & & \\ -1 & & \\ & & 1 \end{bmatrix} & (l = 2) \end{aligned}$$

Decomposition



$$3 \otimes 3 = 1 \oplus 3 \oplus 5$$

* $\oplus$ here represents the direct sum (concatenation in CS terms), i.e. making one continuous list from all elements

Some of the “Types” in e3nn

parity	L (degree)	dimensions	name	declaration
even	0	1	scalar	“0e”
odd	0	1	pseudo scalar	“0o”
even	1	3	pseudo vector	“1e”
odd	1	3	vector	“1o”
even	2	5	-	“2e”
odd	2	5	-	“2o”

Operations

The rotation is given by **Wigner D-matrices** (D for *Darstellung* - representation) and represents the rotation **R acting on the features**, e.g. for $l = 2$:

$$[\lambda_5 \dots \lambda_9]^T \rightarrow \mathcal{D}^{(2)}(\mathbf{R})[\lambda_5 \dots \lambda_9]^T$$

The tensor product of two spherical tensors is generally not a spherical tensor, therefore it has to be decomposed (the most computationally expensive step). The tensor product $\vec{S}^{(l_1)} \otimes \vec{T}^{(l_2)}$ decomposes into:

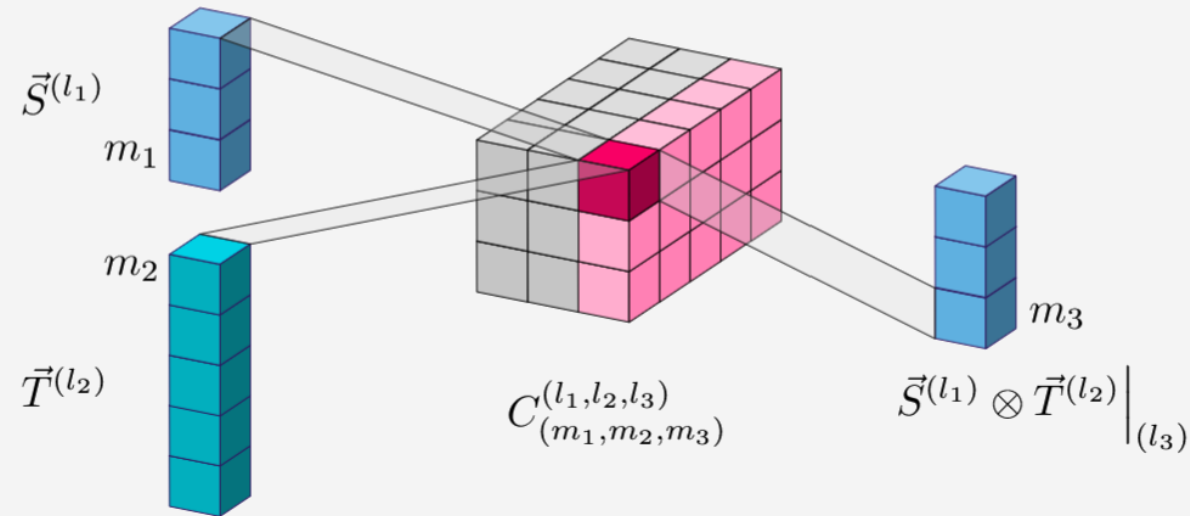
$$l_1 \otimes l_2 = |l_1 - l_2| \oplus |l_1 - l_2 + 1| \oplus \dots \oplus (l_1 + l_2 - 1) \oplus (l_1 + l_2)$$

Therefore, the result is $l_1 + l_2 - |l_1 - l_2| + 1$ new spherical tensors. Each valid combination of (l_1, l_2, l_3) is a tensor **path** (l_3 is the type of output tensor).

The coefficients of the decomposition are given by the **Clebsch-Gordan coefficients**, e.g. $C_{(m_1, m_2, m_3)}^{(l_1, l_2, l_3)}$ is the weight of the product of component m_1 of type l_1 with component m_2 of type l_2 that makes component m_3 of type l_3 .

$$C^{(l_1, l_2, l_3)} \in \mathbb{R}^{(2l_1+1) \times (2l_2+1) \times (2l_3+1)}$$

Operations



In this example, $l_1 = 1, l_2 = 2, l_3 = 1$. This means that $C^{(l_1, l_2, l_3)}$ has $(2l_1 + 1)(2l_2 + 1)(2l_3 + 1) = 45$ components in total. The element $C_{(m_1, m_2, m_3)}^{(l_1, l_2, l_3)}$ (red) encodes how to weight the product of m_1 and m_2 for m_3 , such that the output transforms as an l_3 tensor. This is only one of 12 terms (pink) that make up m_3 here.

I don't find this visualization particularly helpful...

Initialization - Transform to Spherical Harmonics

Real spherical harmonic:

$$Y_m^l: S^2 \rightarrow \mathbb{R}$$

$$\vec{Y}^{(l)} = (Y_{-l}^l, \dots, Y_{l-1}^l, Y_l^l): S^2 \rightarrow \mathbb{R}^{2l+1}$$

Let $\hat{v} = (x, y, z)$ be a point on the unit sphere $S^2 \subset \mathbb{R}^3$. It stems from the normalized vector $\vec{v}$ via $\hat{v} = \frac{\vec{v}}{\|\vec{v}\|}$ for numerical stability reasons. The radial part $\|\vec{v}\|$ can be rescaled and used as a scalar feature. Initialization of the spherical harmonics is done via:

$$Y_0^{(0)}(\hat{v}) = c_0$$

$$Y_{-1}^{(1)}(\hat{v}) = c_1 y, \quad Y_0^{(1)}(\hat{v}) = c_1 z, \quad Y_1^{(1)}(\hat{v}) = c_1 x,$$

$$Y_{-2}^{(2)}(\hat{v}) = c_2 xy, \quad Y_{-1}^{(2)}(\hat{v}) = c_2 yz, \quad Y_0^{(2)}(\hat{v}) = \frac{c_2}{2\sqrt{3}}(2z^2 - x^2 - y^2),$$

$$Y_1^{(2)}(\hat{v}) = c_2 xz, \quad Y_2^{(2)}(\hat{v}) = \frac{c_2}{2}(x^2 - y^2),$$

where c_0, c_1, c_2 are constants and depend on the choice of normalization.

Rotation Equivariance

Equivariance is kept - rotation and then change of basis is the same as change of basis and then rotation.

Rotation of the tensor before transforming
it into spherical harmonics

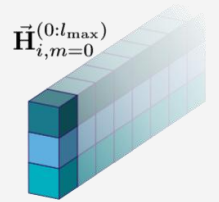
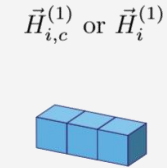
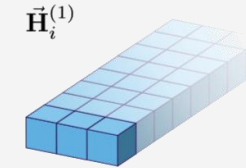
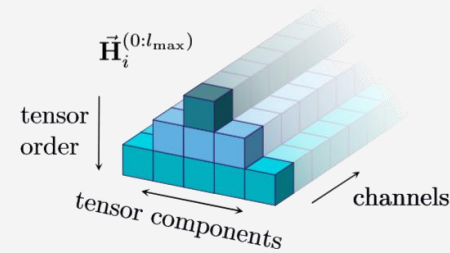
$$\vec{Y}^{(l)}(\mathbf{R}\hat{v}) = \underbrace{\mathcal{D}^{(l)}(\mathbf{R})}_{\text{rotation in the "language" of spherical harmonics}} \underbrace{\vec{Y}^{(l)}(\hat{v})}_{\text{spherical tensor}}$$

rotation in the "language" of
spherical harmonics

Spherical EGNN - Example

1. Initialize node
(possibly also edge)
features

$$\vec{H}_i^{(0:l_{max})} := \bigoplus_{l=0}^{l_{max}} \vec{H}_i^{(l)} = \begin{bmatrix} \vec{H}_i^{(0)} \\ \vdots \\ \vec{H}_i^{(l_{max})} \end{bmatrix}$$



2. Split vectors into
radial and angular
parts

$$x_{ij} = \|\vec{x}_{ij}\|$$

$$\hat{x}_{ij} = \frac{\vec{x}_{ij}}{\|\vec{x}_{ij}\|}$$

3. Project the
directional part to
spherical harmonics

$$\vec{Y}^{(l)}(\hat{x}_{ij})$$

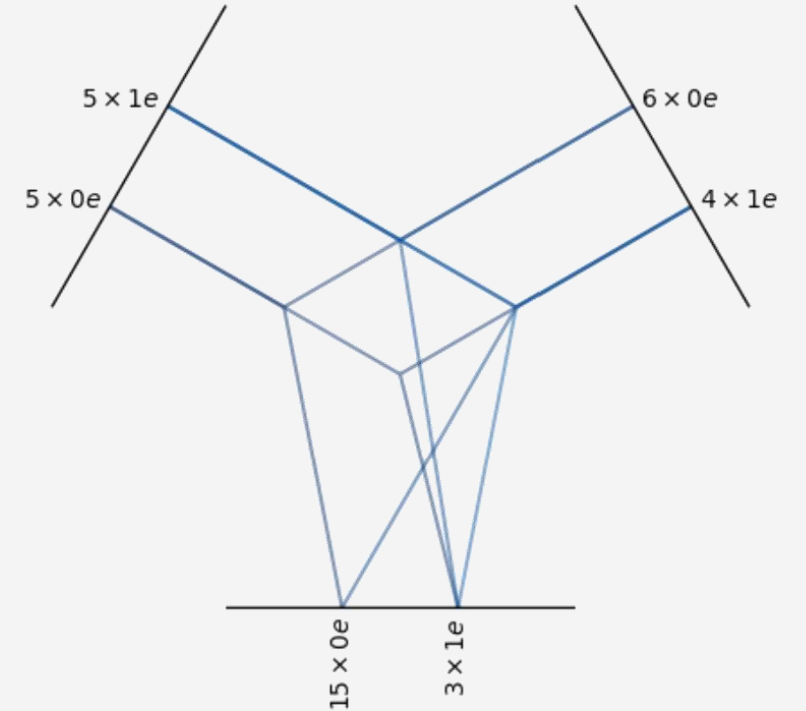
4. Map the range of
 x_{ij} from $[0, \infty)$ to
 $[-1, 1]^{n_b}$ where n_b
is the number of
“basis functions”

*“basis functions” here refers to mappings like the RBF or the Bessel functions for example (the nomenclature is just confusing)

Spherical EGNN - Example

5. Parameterize the tensor product by assigning a learnable weight to each output spherical tensor

$$\begin{aligned}\vec{S}^{(l_1)} \otimes \vec{T}^{(l_2)} &= \bigoplus_{l_3 = |l_1 - l_2|}^{l_1 + l_2} w_{l_1, l_2, l_3} \vec{S}^{(l_1)} \otimes \vec{T}^{(l_2)} \Big|_{(l_3)} \\ &= \bigoplus_{l_3 = 0}^{l_3 = l_{\max}} \left(\sum_{(l_1, l_2 \rightarrow l_3)} w_{l_1, l_2, l_3} \vec{S}^{(l_1)} \otimes \vec{T}^{(l_2)} \Big|_{(l_3)} \right)\end{aligned}$$



e.g.:

For 0e: $(5 * 6 + 5 * 4) * 15 = 750$ paths (and weights)

For 1e: $(5 * 4 + 5 * 6 + 5 * 4) * 3 = 210$ paths (and weights)

Total: $750 + 210 = 960$ weights

*This can also be performed between each valid degree-channel pair if >1 channels are available →

fully-connected tensor product

Spherical EGNN - Example

6. Graph-convolutional filter, then permutation invariant aggregation (e.g. sum-aggregation) for the messages

$$Filter(\vec{x}_{ij}, \vec{H}_j^{(0:l_{max})}) = \vec{Y}^{(0:l_{max})}(\hat{x}_{ij}) \otimes \vec{H}_j^{(0:l_{max})}$$

$$\vec{M}_i^{(0:l_{max})} = \sum_{j \in \mathcal{N}_i} Filter(\vec{x}_{ij}, \vec{H}_j^{(0:l_{max})})$$

7. Channel mixing

$$Mix(\vec{M}_i^{(0:l_{max})}) = \bigoplus_{l=0}^{l_{max}} w_{c'c}^{(l)} \vec{M}_{i,c}^{(l)}$$

8. Component-wise non-linearity via non-linear **gating** (i.e. apply non-linearity to scalars and then multiply with the tensor)

$$Gated - NL(\vec{M}_i^{(0:l_{max})}) = \eta_0(\vec{M}_i^{(0)}) \oplus \left(\bigoplus_{l=1}^{l_{max}} \eta_l(Gate(\vec{M}_i^{(0)})) \vec{M}_i^{(l)} \right)$$

Spherical EGNN - Example

9. Equivariant
message passing

$$\text{Upd} \left(\vec{H}_i^{(0:l_{\max})}, \vec{M}_i^{(0:l_{\max})} \right) = \vec{H}_i^{(0:l_{\max})} + \text{Gated} - \text{NL} \left(\text{Mix} \left(\vec{M}_i^{(0:l_{\max})} \right) \right)$$

10. Update the node
representations

$$\vec{H}_i^{(0:l_{\max})} = \text{Upd} \left(\vec{H}_i^{(0:l_{\max})}, \vec{M}_i^{(0:l_{\max})} \right)$$

Unconstrained Geometric GNNs

Don't bake symmetries into their architectures.

The model learns approximate symmetries through loss functions, data augmentation or global / local canonicalization, i.e. soft constraints -> ***a data-driven view of symmetries***.

Viability will depend on the problem and the data at hand.

Some successful models:

Duval, A. et al. **arXiv:2305.0557**, Dosovitskiy, A. et al. **arXiv:2010.11929v2**, Gerken, J. E. et al. **arXiv:2202:03990**, Hu, W. et al. **arXiv:2103.01436**, Sanchez-Gonzalez, A. et al. **arXiv:2002:09405**, Qi, C. R. et al. **NIPS 2017**, Zitnick, C. L. et al. **arXiv:2206.14331**

Good Sources

GNNs:

<https://www.nature.com/articles/s43586-024-00294-7>

https://projects.volkamerlab.org/teachopencadd/talktorials/T035_graph_neural_networks.html

Geometric DL books:

<https://hal.science/hal-04409413/file/Hitchhiker%27s%20Guide.pdf> # heavily used in these slides

<https://arxiv.org/pdf/2104.13478> # Geometric Deep Learning

<https://arxiv.org/pdf/2512.13927> # make an equivariant transformer from scratch

Invariant GNN showcase:

https://projects.volkamerlab.org/teachopencadd/talktorials/T036_e3_equivariant_gnn.html

Good Sources

Geometric DL:

<https://dmol.pub/index.html> # course on DL for molecules and materials

<https://e3nn.org/> # e3nn docs, make sure to read all of them

https://blondegeek.github.io/e3nn_tutorial/#reading # e3nn tutorials (old version!)

<https://www.youtube.com/watch?v=q9EwZsHY1sk> # e3nn tutorials (old version!)

<https://www.youtube.com/watch?v=z2OEyUgSH2c&list=PL8FnQMH2k7jzPrxqdYufoiYVHim8PyZWd&index=1&p=p=iAQB> # geometric DL lectures from Uni Amsterdam

https://www.youtube.com/watch?v=xTi1w2t_TFE # great lecture by Petar Velickovic and Michael Bronstein, highly recommend!